# A Verifiable Environment for Neural Architecture Design

**Anonymous Author(s)**
Affiliation
Address
email

## Abstract

Reinforcement learning and agent evaluation both depend on a *verifier*: a function that scores a candidate cheaply and without a human. Code and mathematics have compilers and unit tests; neural architecture design has had no public verifier. We present an environment in which an agent designs a neural network by emitting edits to a typed, structured graph, and a deterministic, sub-millisecond verifier grades the result against structural, budget, and serving-physics constraints – no human or LLM in the scoring loop. The environment ships a procedural task generator (ten families, satisfiability and non-vacuity proofs, red-teamed anti-gaming constraints), a difficulty-calibration harness, and a grounding study: across $264$ architectures, verifier blockers predict PyTorch forward-pass failure at $96/96$, and we report honestly that the 0–100 health score is a validity margin, not a quality ranking. Three findings follow. *As a repair signal*, one round of verifier feedback lifts `claude-sonnet-4-6` from $79\%$ to $100\%$ and `deepseek-chat` from $59\%$ to $90\%$. *As a data foundry*, fine-tuning a 1.5B model on 3,000 minted verified pairs lifts strictly-unseen pass@1 from $17\%$ to $76\%$, and $100$ GRPO steps on top reach $80\%$ ($n$=192); a contamination audit we ran on ourselves, disclose, and ship shows the naive protocol would have reported $86\%$, quantifying the memorization share. *As ground truth*, eleven LLM reward models score $0\%$ false-positive on blatantly broken designs yet approve 26–47$\%$ of near-miss designs one edit from correct, failing exactly where reward hacking lives; the verifier is $0\%$ on both tiers. The same pure function is a leaderboard, an RL gym, and a verified-data generator; we release all of it.

**Reproducibility note.** All results marked *measured* are reproduced by the commands in the repository; the calibration, amplification, and reward-model results use provider API keys, the rest need neither keys nor GPUs. `VERIFICATION.md` in the repository maps each headline number to its source artifact or reproduction command. Code and data: anonymized repository in the supplementary material; released under MIT upon publication.

## 1 Introduction

The scarce ingredient in modern agent training and evaluation is not the policy but the verifier. SWE-Bench [1] grades a code patch by running the project's own test suite; $\tau$-bench [2] grades a tool-using dialogue by diffing the resulting database state against a goal; in mathematics, programmatic answer checking underpins both benchmark grading and verifier training [14]. None of these requires a human judge or a second model, and this is precisely why they are trusted and why they can be used as training signals.

Neural architecture design has lacked such a verifier, for a structural reason: whether a proposed network is even runnable (attention head counts that divide the embedding dimension, linear widths that match upstream shapes, a graph whose input reaches its output, a parameter count inside a budget) is a pure function only when architectures are represented as typed, structured graphs rather than free-form code. Coding agents emit code, not graphs, so their output carries no shape-level verifiability. We close this gap by building the environment around a typed graph and the deterministic checks that a production editor already runs on it. Existing structured views of architectures, such as Netron [13], are read-only viewers: they render a graph but do not edit it, propagate shapes, or verify, and so cannot serve as an action space or a reward.

Our contributions are: (1) a design-from-spec and edit-in-place environment with a sub-millisecond programmatic verifier; (2) a procedural task generator with satisfiability proofs, non-vacuity proofs, and anti-gaming constraints; (3) a calibration harness that measures difficulty against the labs' usable band, with keyless self-tests that bracket the harness itself; (4) a grounding study connecting verdicts to real PyTorch behavior, reported with its negative results intact; and (5) a set of empirical results released with the environment: a verifier-in-the-loop lift (79 → 100, one repair round, replicated on a second model), a strictly-held-out lift from $17\%$ to $76\%$ by SFT on environment-minted verified pairs and to $80\%$ with GRPO on top ($n$=192) (with a disclosed contamination audit that quantifies the memorization share of the naive protocol, and the RL-from-raw null reported and explained), and an LLM-reward-model audit whose $0\%$ false-positive rate on blatant defects collapses to 26–47% on near-misses – atop a leaderboard, an RL gym, and a verified-data foundry.

## 2 The environment

**State.** A typed graph of a neural network: components drawn from the production editor's vocabulary of 182 layer types, each carrying parameters, connected by directed edges; the released grader computes serving physics for the structurally load-bearing subset (linear, convolutional, embedding, normalization, and the attention family). **Action.** A batch of structured edits from a fixed vocabulary: `add_component`, `add_connection`, `update_params`, `delete_component`, `replace_model`, and others, the same vocabulary a production editor executes. **Reward.** A grading function returning a 0–100 health score plus a list of hard blockers; Figure 1 shows the full loop.

**Formalization.** The environment is a deterministic single-agent MDP $(\mathcal{S}, \mathcal{A}, T, R)$. A state $s \in \mathcal{S}$ is a typed graph $(V, E)$: nodes $V$ each carry a type and a parameter dictionary, edges $E$ are directed. An action $a \in \mathcal{A}$ is a batch of typed edits from the fixed vocabulary; the transition $T(s, a)$ applies them, with no environment stochasticity. The reward is the grader's health score $R(s') \in [0, 100]$, and a task's success predicate is $R(s') \geq \tau \wedge \neg\,\mathrm{blocked}(s') \wedge \phi(s')$, where $\phi$ collects the task's machine-checkable constraints (required types, parameter/KV/latency budgets, action-economy limits). Since $T$ and $R$ are pure functions, an episode is fully reproducible from its seed, and the same $R$ is at once an evaluation metric, an RL reward, and a data-generation filter.

**Widths, parameters, and cost.** Each node type defines an output width $w_{\text{out}}$ and a parameter count from its typed parameters. For the load-bearing types,

$$\mathrm{linear}(d_{\text{in}}, d_{\text{out}}): \quad w_{\text{out}} = d_{\text{out}}, \quad |\theta| = d_{\text{in}}\, d_{\text{out}} + d_{\text{out}}, \tag{1}$$

$$\mathrm{attention}(d, H): \quad w_{\text{out}} = d, \quad |\theta| = 4d^2 + 4d, \tag{2}$$

$$\mathrm{conv}(c_{\text{in}}, c_{\text{out}}, k): \quad w_{\text{out}} = c_{\text{out}}, \quad |\theta| = c_{\text{in}}\, c_{\text{out}}\, k^2 + c_{\text{out}}. \tag{3}$$

Total parameters are $\textsc{Params}(G) = \sum_{v \in V} |\theta(v)|$, and the decode-time KV-cache cost per token is

$$\textsc{Kv}(G) \;=\; \sum_{v \,\in\, \mathrm{attn}(G)} 2\, H_v^{\mathrm{KV}}\, \frac{d_v}{H_v}\, b, \tag{4}$$

for $b$ bytes per cached value (multi-head attention is the case $H_v^{\mathrm{KV}} = H_v$; grouped-query attention shrinks $H_v^{\mathrm{KV}}$). A task with constraints $\phi$ and threshold $\tau$ is solved iff

$$\textsc{Pass}(T, G) \;=\; \big[\, B(G) = \emptyset \,\big] \;\wedge\; \big[\, h(G) \geq \tau \,\big] \;\wedge\; \phi(G), \tag{5}$$

where $B(G)$ is the hard-blocker set of Algorithm 1, $h(G)$ its health score (whose soft-penalty weights are implementation detail: every result in this paper depends only on the success predicate, and the score is diagnostic), and $\phi(G)$ conjoins the machine-checkable constraints: required

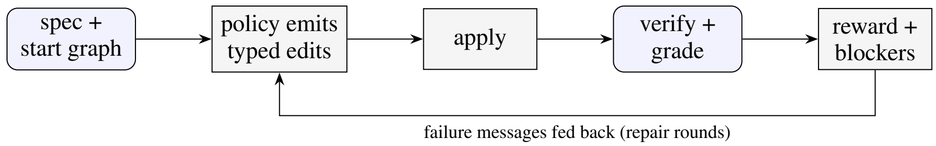


Figure 1: The environment loop. A policy edits a typed graph; a deterministic, sub-millisecond grader returns a reward and hard blockers. Feeding the blocker messages back for repair rounds is the amplification setting (Section 6); with no feedback it is single-shot.

**Algorithm 1** GRADE$(T, G)$: the deterministic verifier

**Require:** task $T$ with constraints $\phi$ and threshold $\tau$; typed graph $G = (V, E)$
**Ensure:** health score $h \in [0, 100]$; hard-blocker set $B$

$B \leftarrow \emptyset$
**if** $\neg$ REACHES(in$(G)$, out$(G)$) **then** $B \leftarrow B \cup \{\text{DISCONNECTED}\}$
**end if**
**for** each attention node $v \in V$ **do**
  **if** $v$.embedDim $\bmod$ $v$.numHeads $\neq 0$ **then** $B \leftarrow B \cup \{\text{HEADDIV}(v)\}$
  **end if**
  **if** $v$.numHeads $\bmod$ $v$.numKVHeads $\neq 0$ **then** $B \leftarrow B \cup \{\text{GQADIV}(v)\}$
  **end if**
**end for**
**for** each node $v$ with typed input width $w_{\text{in}}(v)$ and parent $u$ **do**
  **if** $w_{\text{in}}(v) \neq w_{\text{out}}(u)$ **then** $B \leftarrow B \cup \{\text{SHAPEMISMATCH}(v)\}$
  **end if**
**end for**
compute PARAMS$(G)$, KV$(G)$, LATENCY$(G)$; add a blocker for each budget in $\phi$ violated, and for each required type absent
$h \leftarrow 0$ **if** $B \neq \emptyset$ **else** $100 - \sum_{s \in \text{SOFT}(G)} \text{pen}(s)$
**return** $(h, B)$

types, a two-sided parameter band $p_{\min} \leq \text{PARAMS}(G) \leq p_{\max}$, KV and latency budgets, and an action-economy limit $|a| \leq a_{\max}$. Every term is a pure function of $G$, so Eq. (5) is decidable in $O(|V| + |E|)$.

The grader composes three checks, each pure and sub-millisecond: a structural score with hard blockers (disconnected input/output, attention divisibility, parameter bloat), a guardrail pass over the proposed edits, and a shape propagator that catches shape bugs an edit would introduce. No LLM grades anything, so there is no persuasion surface and no stochasticity in the reward.

**The checks.** The grader runs three families of checks, in order. (i) *Structural blockers*: a disconnected input or output; an attention layer whose `embedDim` is not divisible by `numHeads` (or `numHeads` not divisible by `numKVHeads` under grouped-query attention); a linear layer whose `inFeatures` do not match the upstream width; a parameter count far over budget. (ii) *Serving physics*: parameters, forward multiply-accumulates, and KV-cache bytes per token, computed with the canonical formula (grouped-query attention caches $2 \cdot \texttt{numKVHeads} \cdot d_{\text{head}} \cdot b$ bytes per token at $b$ bytes per value; multi-head attention is the special case $\texttt{numKVHeads} = \texttt{numHeads}$), and checked against per-task parameter, KV, and decode-latency budgets. (iii) *Shape propagation*: a symbolic forward pass over the typed graph that catches interface mismatches an edit would introduce before any tensor is allocated. Each pass is $O(|V| + |E|)$ over the graph and completes in well under a millisecond, which is what makes the reward usable as an inner-loop RL signal rather than an offline evaluation.

A task pairs a natural-language specification with a starting graph and machine-checkable constraints, e.g. *"This encoder fails validation: embedDim (192) is not divisible by numHeads (7). Repair the attention configuration in place with at most 2 actions. Do not rebuild the model from scratch."* Figure 2 shows this state and the blocker the verifier returns.

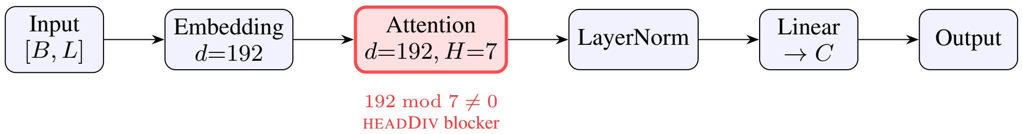


Figure 2: A task state as a typed graph, and a hard blocker the verifier returns in $O(1)$ at the attention node: embedDim=192 is not divisible by numHeads=7 (Algorithm 1, line 5). Because widths and divisibility are typed-node properties, this check is a pure function; on free-form code it would require executing the model. The input carries shape $[B, L]$ ($B$ batch, $L$ sequence length), widening to $[B, L, 192]$ after the embedding. The repair task asks the policy to fix exactly this in $\leq 2$ edits without rebuilding.

| Failure category (curated split, `claude-sonnet-4-6`) | count |
|---|---|
| missing required layer type (e.g. attention) | 3 |
| disconnected graph (input does not reach output) | 2 |
| health score below threshold | 1 |
| over parameter budget | 1 |
| **passed** | **7 / 12** |

Table 1: First-try failure reasons of a frontier model on the curated production-architecture split: 5 of 12 tasks fail, and a failed task can trigger more than one detected reason (7 reasons across the 5 failures). Every reason is detected deterministically by the verifier.

## 3 Task generation

Curated tasks are a seed. A deterministic generator mints splits of any size from a seed across ten families: six design-from-spec (dense classifier, autoencoder, convolutional classifier, transformer encoder, grouped-query attention encoder, two-tower retrieval) and four edit-in-place (repair a broken attention configuration, trim an oversized model under a parameter budget, grow an undersized model into a two-sided parameter band, insert normalization). Because tasks are synthesized rather than scraped, a held-out split need never appear on the public web; contamination resistance against *model pretraining* is structural (unverified against a web index), and train/eval separation *within* the environment is enforced by an explicit holdout exclusion after our audit (Section 7).

**Satisfiability and non-vacuity (measured).** Every generated task carries its own reference solution. A 500-case sweep asserts that each reference passes its own task (no unsatisfiable tasks), and every edit-in-place start graph is asserted to fail its own task untouched (no vacuous tasks). Both run in CI without keys.

**Anti-gaming.** Edit-in-place families forbid `replace_model` and `clear_canvas`, so an agent that cannot diagnose a defect cannot pass by regenerating the whole network; budgets are two-sided bands rather than ceilings, so "delete everything" fails a trim task; a KV-cache budget is always paired with a required-attention constraint, so amputating attention cannot zero the cache. Nine such strategies, their defenses, and the pinned tests that fail if any defense is weakened are enumerated in the repository's red-team report (`ROBUSTNESS.md`). An LLM-driven task proposer accepts nothing without a satisfiability proof: the proposer's own reference must pass the grader under always-enforced safety-net constraints, so LLM authorship, unsafe for human- or LLM-judged benchmarks, is safe here.

**A benchmark that bites (measured).** On twelve hand-authored curated tasks built from real production architectures (a CIFAR CNN, a DLRM click-through-rate ranker, a Behavior Sequence Transformer, a two-tower retrieval encoder, a semantic-search encoder, and repair/scaling tasks), `claude-sonnet-4-6` passes 7/12 first try (mean health score 76); Appendix A sketches four of the tasks as typed graphs. The failures are machine-checkable and mundane (Table 1): it omits the required attention layer, leaves the graph disconnected, or blows the parameter budget by an order of magnitude. A benchmark a frontier model clears 12/12 measures nothing.

| Family | single-shot | with verifier feedback |
|---|---|---|
| dense classifier | 100% | 100% |
| autoencoder | 100% | 100% |
| convolutional classifier | 83% | 100% |
| GQA encoder | 100% | 100% |
| repair attention | 100% | 100% |
| trim under budget | 100% | 100% |
| grow to band | 100% | 100% |
| two-tower retrieval | 100% | 100% |
| **transformer encoder** | **8.3%** | **100%** |
| **insert normalization** | **0%** | **100%** |
| **overall** | **79%** | **100%** |

Table 2: Per-family pass rate for `claude-sonnet-4-6` on the public benchmark (generator v2; $n$=12 per family, all 120 tasks graded with zero exclusions). Seven families sit at 100% and the convolutional classifier at 83%; the two the model finds hard are exactly where verifier-in-the-loop feedback (second column; Section 6) pays off. Overall single-shot 79% (Wilson 95% CI $[71, 85]$) rises to 100% ($[97, 100]$); the v1-generator measurement (82% $\rightarrow$ 100%, $n$=117) replicates within three points.

## 4 Difficulty calibration

An ungameable environment is still useless at 0% pass (no learning signal) or near 100% (nothing to learn). Practitioners cite a floor near 2–3% pass rate on the hard end (roughly one success per 33–50 rollouts) for a usable RL environment. The calibration harness measures per-family pass rates with Wilson 95% intervals [15] and flags each family's band. Two keyless self-test policies bracket the harness itself: a `reference` policy that replays known-good solutions (measured: 100% pass) and a `noop` policy that submits empty plans (measured: 0% pass). If either self-test deviates, the harness rather than the model is broken, and CI fails.

**Measured (`claude-sonnet-4-6`, public benchmark, $n = 12$/family).** Single-shot pass rate per family: most families saturate at or near 100% (a curriculum tier for a frontier model) and two are hard: transformer encoder (8.3%) and insert-norm (0%) (Table 2). Deterministic grading, reproducible from the seed.

## 5 Grounding study

Does the verifier's verdict correspond to reality? We generated clean reference architectures and systematically corrupted variants (broken attention divisibility, mismatched linear width, a severed mid-graph edge), built every graph as a PyTorch module, and trained each briefly on a synthetic fixed-teacher task.

**Result (measured, 264 graphs, two seeds).** The 264 graphs comprise three classes: 80 clean references, 96 the verifier blocked, and 88 deliberately width-corrupted. Verifier blockers predicted a forward-pass failure with perfect precision: 96 of 96 blocked graphs failed to run. The clean graphs all constructed (80 of 80) and made training progress in 90%. The third class quantifies the transparent rubric's one systematic miss: it does not chase linear widths through the graph, so all 88 width-corrupted graphs pass the rubric yet fail the forward pass (88/88); the richer verifier in the production system flags this class. We therefore claim the blocked side (a blocker proves runtime failure) and report the blind spot as a measured limitation rather than hiding it (Figure 3).

**Negative result (measured).** Within clean graphs, the 0–100 score's magnitude does *not* rank training quality: Spearman correlation between the score and relative loss decrease was $-0.58$, because deeper, more structured models make slower per-step progress on the short synthetic probe than tiny ones. The score is therefore a validity margin, not a quality ranking. A learned quality head, trained on (architecture, verdict, real training curve) triples that the environment can mint at

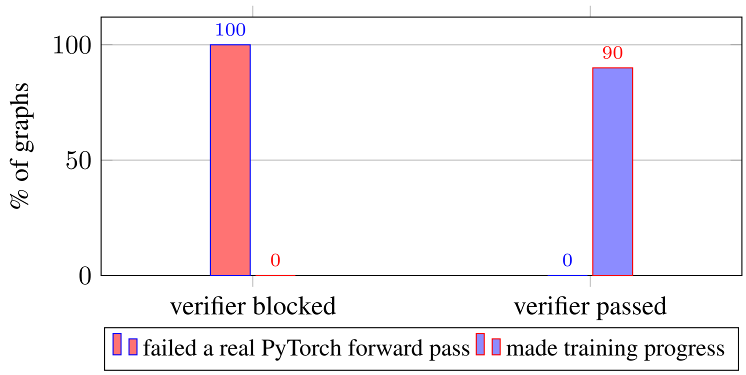


Figure 3: Grounding study (264 graphs, two seeds: 80 clean, 96 blocked, 88 width-corrupted). Every graph the verifier blocked (96/96) failed a real PyTorch forward pass; every clean graph passed and constructed (80/80), with 90% making training progress. The width-corrupted class (not shown) passes the transparent rubric yet fails 88/88 forward passes – the rubric's blind spot, measured in the text.

**Algorithm 2** ROLLOUT$(T, \pi, k)$: verifier-in-the-loop repair

**Require:** task $T$; policy $\pi$; maximum repair rounds $k$
**Ensure:** pass $\in \{0, 1\}$; rounds used

1: $G \leftarrow T$.startGraph; $\quad m \leftarrow \varnothing$ ▷ $m$: verifier feedback
2: **for** $t \leftarrow 1$ **to** $k$ **do**
3: $\quad a \leftarrow \pi\big(T.\text{spec},\ \text{SERIALIZE}(G),\ m\big)$ ▷ batch of typed edits
4: $\quad G \leftarrow \text{APPLY}(G, a)$
5: $\quad (h, B) \leftarrow \text{GRADE}(T, G)$
6: $\quad$ **if** $B = \emptyset \ \wedge\ h \geq \tau \ \wedge\ \phi(G)$ **then return** $(1, t)$
7: $\quad$ **end if**
8: $\quad m \leftarrow \text{EXPLAIN}(B)$ ▷ the verifier's failure messages
9: **end for**
10: **return** $(0, k)$

scale, is the natural next step and the point at which the verifier would become a calibrated predictor rather than a gate.

## 6 Verifier-in-the-loop amplification

Single-shot is the $k$=1 special case (no feedback consumed); the amplification setting sets $k$=3. Algorithm 2 states the loop; the only difference between the two arms is whether line 8's feedback $m$ re-enters line 3. Because grading is free, the verifier's failure messages can be fed back to a policy for repair rounds. We measure, per provider, the pass rate of a single shot versus the pass rate when up to $k$ repair rounds are allowed, holding tasks, model, and prompt fixed so that the only variable is the feedback loop. The framing is deliberately pro-model: the environment's value is the lift it adds to any model.

| Model | single-shot pass | with verifier feedback | lift |
|---|---|---|---|
| claude-sonnet-4-6 | 79% | **100**% | +21 pts |
| deepseek-chat (V3) | 59% | **90**% | +31 pts |

Table 3: Verifier-in-the-loop lift, per provider. Same tasks, model, and prompt; the only variable is the feedback loop.

The lift is concentrated on the families a frontier model finds hard (Figure 4): of the 25 tasks feedback rescues, the transformer encoder (8.3% $\rightarrow$ 100%) accounts for 11 and insert-norm (0% $\rightarrow$ 100%) for 12, with the convolutional classifier (83% $\rightarrow$ 100%) contributing the remain-

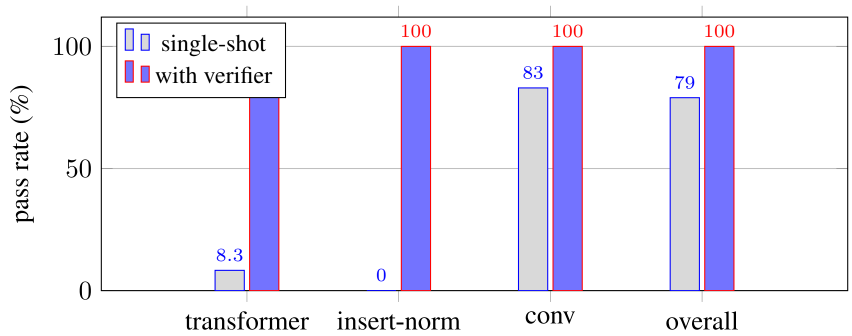


Figure 4: Verifier-in-the-loop lift for `claude-sonnet-4-6`. The overall gain (79 $\rightarrow$ 100; 25 tasks rescued) is concentrated on the two families the model finds hard (11 fixes on the transformer encoder, 12 on insert-norm), with the convolutional classifier contributing the remaining 2; the seven families already at 100% single-shot are omitted.

| max repair rounds $k$ | claude-sonnet-4-6 | deepseek-chat |
|---|---|---|
| $k = 1$ (single-shot) | 79% $[71, 85]$ | 59% $[47, 71]$ |
| $k = 2$ | **100**% $[97, 100]$ | **90**% $[80, 95]$ |
| $k = 3$ | 100% $[97, 100]$ | 90% $[80, 95]$ |

Table 4: Ablation on the number of verifier-feedback repair rounds, two models (generator v2; claude $n$=120, deepseek $n$=59; Wilson intervals). For *both*, a single repair round ($k$=2) provides the entire lift – every task the verifier can help fix is fixed after one round of its failure messages – and $k$=3 adds nothing. The pattern is not model-specific.

ing 2; the seven other families are already at 100% single-shot. A single repair round closes the whole gap (Table 4). Numbers are from generator v2 with all 120 tasks graded, and they *replicate* the v1 measurement (82% $\rightarrow$ 100%, $n$=117) within three points. The effect is not model-specific: on the same tasks, a second, weaker model (deepseek-chat) goes 59% $\rightarrow$ 90% (+31 points, $n$=59; v1: 60 $\rightarrow$ 88), with every fix again landing at $k$=2 and none at $k$=3. The weaker the model, the more the verifier is worth.

## 7 Training on the environment

The environment trains models through two channels, and we report both honestly.

**Verified SFT: the data foundry works.** Every generated task carries a reference solution that provably passes the grader, so the environment mints unlimited verified (spec $\rightarrow$ edits) supervised pairs, keylessly and for free. Fine-tuning Qwen2.5-1.5B-Instruct [17] on 3,010 such pairs (LoRA [16], one free T4, under ten minutes), with every training row verified to share no task with the evaluation split, lifts strictly-held-out pass@1 from 17.2% (33/192, Wilson $[13, 23]$) to 76.0% (146/192, $[70, 82]$), and 100 GRPO steps atop the SFT checkpoint reach **80.2**% (154/192, $[74, 85]$; Table 5, Figure 5). An independent rerun of the GRPO stage from the same SFT checkpoint reproduces the headline exactly (154/192), while the secondary metrics move within run-to-run noise (parse failures 10 vs. 14, mean reward 1.12 vs. 1.10), so we treat the pass@1 gain – not the parse-failure reduction – as the replicated finding. The RL increment is directionally consistent across four independent runs (+9.4, +3.1, +4.2, +4.2 points), and the SFT level replicates across retrains (76.0% and 77.1% at $n$=192; 68.8% at $n$=64). The surviving failures are substantive design errors the verifier pinpoints (disconnected graphs, too few components), not formatting.

**An audit, disclosed.** These are the *corrected* numbers, and the correction is part of the contribution. Our own contamination audit found that the original sampler drew parameters from small discrete lists, yielding only 127 distinct tasks – so despite disjoint seeds, every task in the original eval split had appeared in training: the naive protocol measured 85.9% ($[75, 92]$) where the honest same-split number is 68.8% (44/64, $[57, 79]$). We widened the sampler to broad ranges (17,462 dis-

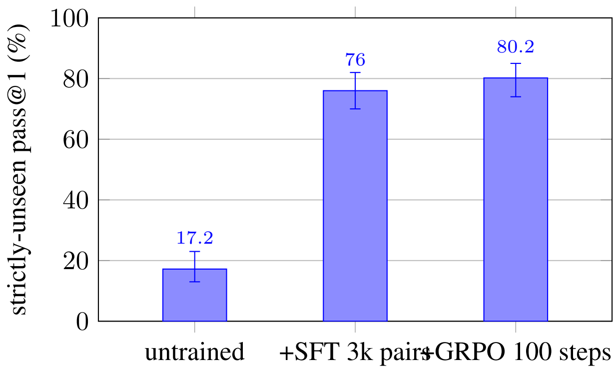


Figure 5: The corrected-protocol checkpoint chain (Qwen2.5-1.5B, seed-999 strictly-unseen split, $n$=192; error bars are Wilson 95% intervals). SFT on 3k environment-minted verified pairs lifts pass@1 from 17.2% to 76.0% with non-overlapping intervals; 100 GRPO steps on the merged SFT checkpoint reach 80.2%, a level an independent GRPO rerun reproduces exactly.

tinct tasks in a 20k draw at seed 999, reproducible keylessly; validated by the released satisfiability sweep), added an explicit holdout exclusion to the data minter (re-verified: 0 of the 192 evaluation tasks appear among the 3,010 training rows), and re-ran the pipeline. The 17-point gap between protocols is itself a measurement – the memorization share [18] of the naive number – and the audit script ships with the environment.

**Scaling, replication, and transfer.** Data scaling is steep and non-monotonic at the low end (Figure 6; corrected zero-overlap protocol, $n$=192): 236 and 474 verified pairs *underperform* the raw instruct model (12.5% and 15.1% vs. 17.2%) – a half-learned edit format displaces the base model's occasional valid outputs before replacing them – while 945 pairs cross above the baseline (26.6%) and 3,010 reach 76.0%; parse failures fall monotonically with data (97 $\rightarrow$ 58 $\rightarrow$ 56 $\rightarrow$ 14 of 192). The same U-shape appeared under the pre-audit protocol (9.4%, 14.1%, 34.4% against a 25.0% baseline), so the non-monotonic onset is a property of the recipe rather than of a particular split; at any single point the dip is within Wilson-interval overlap, and what replicates across the two independent protocols is the shape – dip, crossover near 1k pairs, steep rise. Transfer tracks data diversity, and the audit gave us a controlled comparison: models fine-tuned on the v1 data (127 distinct tasks) left the twelve hand-authored curated tasks unchanged (3/12, the untrained level), while the same recipe on the diverse v2 data doubles them to 6/12 ($n$=12, suggestive rather than conclusive; surviving failures are substantive, e.g. missing attention). Diversity is the operative lever for out-of-distribution transfer, and the generator makes it a parameter rather than a labeling effort.

**RL alone at small scale: an honest null, now explained.** The released GRPO [6] loop runs end-to-end against the reward server on the same hardware, but RL *from the raw instruct model* does not move held-out metrics: two runs (learning rates $10^{-6} \times 150$ and $10^{-5} \times 250$ steps, every checkpoint evaluated and reported) sit at or below their own baselines. The SFT result explains the failure: the raw policy cannot emit parseable edits on a third of tasks, so the group-relative policy gradient starves. This is the classic SFT-then-RL split – supervised learning teaches the action format, RL then refines – and the environment supplies both stages: the verified SFT data and the deterministic reward. Completing the recipe confirms it: under the corrected protocol, 100 GRPO steps from the (merged) SFT checkpoint lift strictly-unseen pass@1 from 76.0% to $\mathbf{80.2}$% ($n$=192), a gain an independent GRPO rerun from the same SFT checkpoint reproduces exactly (154/192 both times); mean reward rises 1.06 $\rightarrow$ 1.12 (1.10 on the rerun), while the parse-failure reduction (14 $\rightarrow$ 10) does not replicate (14 on the rerun) and we do not claim it. The pre-audit chain showed the same pattern (79.7 $\rightarrow$ 89.1%). The same loop that was inert on the raw policy is productive once the policy can act, and every surviving failure is a substantive design error (disconnected two-tower graphs, component-count misses), not formatting.

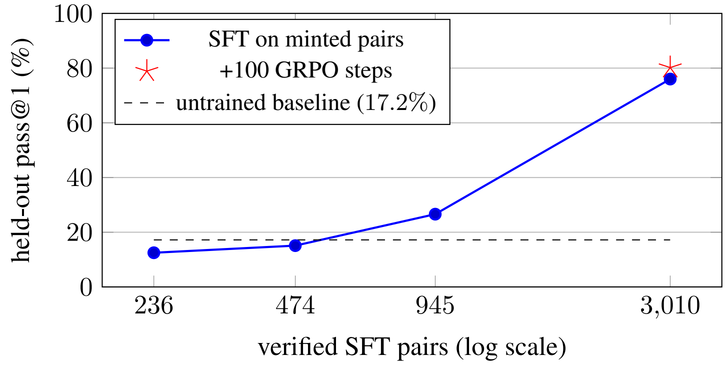


Figure 6: Held-out pass@1 vs. environment-minted verified SFT pairs (Qwen2.5-1.5B, LoRA, 2 epochs; corrected zero-overlap protocol, seed-999 split, $n$=192). The x-axis is the number of verified rows actually trained on (nominal mints of 250/500/1000/3000 minus rows colliding with the holdout). The curve is steep and non-monotonic: below $\sim$1k pairs the half-learned edit format underperforms the untrained model (17.2%, dashed); 945 pairs cross above (26.6%); 3,010 reach 76.0%. Pairs are free to mint, so the x-axis costs seconds of generation. The star marks 100 GRPO steps on the 3k checkpoint (80.2%).

| | pass@1 | parse failures | mean reward |
|---|---|---|---|
| *corrected protocol (v2, verified zero train/eval overlap, $n$=192)* | | | |
| Qwen2.5-1.5B (untrained) | 17.2% [13, 23] | 76/192 | 0.19 |
| + SFT on 3k clean pairs | 76.0% [70, 82] | 14/192 | 1.06 |
| + GRPO after SFT (100 steps) | **80.2%** [74, 85] | **10/192** | **1.12** |
| *pre-audit protocol (v1; eval tasks seen in training)* | | | |
| Qwen2.5-1.5B (untrained) | 23.4% [15, 35] | 19/64 | 0.35 |
| + SFT on 3k pairs | 85.9% [75, 92] | 2/64 | 1.19 |
| + GRPO after SFT | 89.1% [79, 95] | 0/64 | 1.26 |

Table 5: Training on environment-minted data (LoRA, one free T4; seed-999 splits, identical eval configuration; corrected block $n$=192, pre-audit block $n$=64). The corrected zero-overlap protocol is primary (one checkpoint chain, $n$=192; an $n$=64 run of the same protocol gives 14.1 $\rightarrow$ 68.8 $\rightarrow$ 71.9%, and an independent SFT retrain lands at 77.1%). The pre-audit block is retained because the gap between protocols measures the memorization share. Supervised fine-tuning on 3,000 verified pairs the generator mints for free lifts pass@1 by 59 points under the corrected protocol (62 pre-audit) with non-overlapping Wilson intervals and nearly eliminates parse failures. GRPO from the raw model is a null at this scale, but after SFT it works: the GRPO row trains from a fresh-session SFT replication (79.7%), within which RL adds +9.4 points and removes every remaining parse failure (see text). An independent rerun of the corrected-protocol GRPO stage reproduces its pass@1 exactly (154/192; parse 14, mean reward 1.10).

## 8 Auditing LLM reward models with the verifier

Frontier labs increasingly use a strong model *as* a reward model to optimize objectives that are not directly verifiable (xAI reported this for Grok 4.1), a practice studied more broadly as LLM-as-a-judge [12], with process reward models [20] the analogous substitute where no programmatic check exists. The known failure mode is that an LLM reward model drifts: it approves answers that are actually broken. In a domain with a verifiable reward that drift is *measurable*. `reward_anchor.mjs` builds a verifier-labeled set (each design provably passes or fails: a reference design passes, its unsolved starting graph fails) and, given an LLM acting as a reward model, reports its agreement with the verifier and, crucially, its **false-positive rate**: how often it approves a design the verifier proves is broken. Across eleven reward models spanning closed-frontier (Claude, Grok) and open-weights (Qwen, Mistral, DeepSeek, Gemma, Llama), the false-positive rate is uniformly 0% (Table 7): not one approves a design the verifier proves broken. The failure mode that corrupts RLVR – rewarding a broken design – does not appear; the only errors are false *negatives* (1.7–13.3%, over-conservative),

| reward model | blatant FP | near-miss FP | near-miss FN |
|---|---|---|---|
| qwen-2.5-72b-instruct | 0.0% | **46.7**% [35, 59] | 1.7% |
| claude-sonnet-4-6 | 0.0% | 33.3% [23, 46] | 5.0% |
| grok-4 | 0.0% | 25.5% [16, 39] | 7.8% |
| deterministic verifier (ours) | 0% | **0**% | **0**% |

Table 6: The same audit with near-miss negatives on generator v2: passing designs one edit from correct ($n$=60 each; grok $n$=51/37 across tiers from provider 503s; corruption mix: dropped component, dropped connection, broken divisibility, each verified to fail). The v1 measurement (FP 22–48%) replicates. The 0% false-positive rate on blatant negatives collapses to 26–47%; the model that was perfect on blatant negatives (Qwen) is the worst here, approving whatever looks plausible. LLM judges fail precisely where reward hacking lives; the verifier does not.

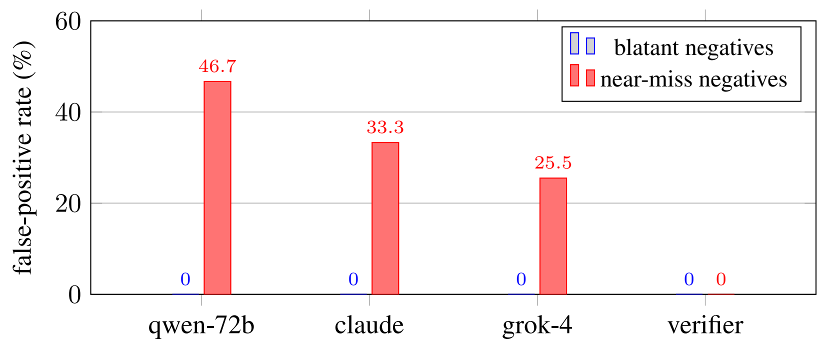


Figure 7: The collapse, visually: LLM reward models that never approve a blatantly broken design approve 26–47% of near-miss designs one edit from correct. The deterministic verifier is 0% on both tiers by construction.

which waste valid designs but do not poison the reward. An LLM reward model here is therefore safe but lossy, and the verifier does the same job with 0% false negatives, deterministically and for free: a verdict costs microseconds of CPU, versus a multi-second, paid, stochastic API round-trip per judgment. This looks reassuring – but the negatives so far are blatant (an unsolved starting graph).

**The** 0% **does not survive near-misses.** We re-run the audit with *near-miss* negatives: passing designs corrupted by a single edit (a dropped connection, an attention head count bumped off divisibility, a dropped component), each re-graded to confirm it fails. On these, the same judges collapse (Table 6, Figure 7): Claude approves 33% of provably-broken designs (Wilson [23, 46]), Grok 26% [16, 39], and Qwen-2.5-72B – among the strongest on blatant negatives (96.7% agreement) – approves 47% [35, 59] while rejecting almost nothing, i.e. it defaults to approval whenever a graph merely looks plausible. The inversion matters: LLM reward models are trustworthy exactly when errors are obvious, and fail exactly where reward hacking [19] lives – subtle defects one edit from correct. The deterministic verifier is 0% on both difficulty tiers by construction. This is a use of the environment beyond training or evaluation: a ground-truth anchor exposing when the LLM reward models labs deploy can and cannot be trusted.

## 9 Verified reasoning traces

The same verifier is a data foundry for the input reasoning-model post-training consumes: (spec $\rightarrow$ reasoning $\rightarrow$ design) triples where the design is re-graded and only passing traces are kept (rejection sampling). No trace enters the set unless its design provably solves the spec. A frontier model's verified yield under this filter – the fraction of its reasoned designs that pass – was 72.5% (327 of 451 tasks, with 49 API failures excluded) on a 500-task run, producing 327 verified traces; the released tool records the yield per run and scales to larger sets. The 327-trace set is released publicly (dataset link withheld for double-blind review; included in the supplementary material). Because the

| model | tier | agreement | false-pos. | false-neg. |
|---|---|---|---|---|
| llama-3.3-70b-instruct | open | 98.3% | **0.0**% | 1.7% |
| qwen-2.5-72b-instruct | open | 96.7% | **0.0**% | 3.3% |
| claude-sonnet-4-6 | frontier | 95.0% | **0.0**% | 5.0% |
| grok-4.5 | frontier | 95.0% | **0.0**% | 5.0% |
| grok-4.20 (reasoning) | frontier | 95.0% | **0.0**% | 5.0% |
| grok-4-fast | frontier | 95.0% | **0.0**% | 5.0% |
| grok-4 | frontier | 94.6% | **0.0**% | 5.4% |
| deepseek-r1 (reasoning) | open | 93.3% | **0.0**% | 6.7% |
| deepseek-chat (V3) | open | 91.7% | **0.0**% | 8.3% |
| mistral-large | open | 88.3% | **0.0**% | 11.7% |
| gemma-2-27b-it | open | 86.7% | **0.0**% | 13.3% |
| deterministic verifier (ours) | — | 100% | **0**% | **0**% |

Table 7: Eleven reward models vs. the verifier on generator v2 ($n$=60 each; grok-4 $n$=37 from provider errors). *Not one* has a false positive (0% across all); the only failure mode is over-conservatism (1.7–13.3% false negative). The three Grok variants agree with the verifier on the same 57/60 examples. The verifier matches the best model (0/0) at zero cost and deterministically. Section text and Table 6 show this 0% does not survive subtler, near-miss negatives.

| | domain | verifiable reward | procedural, contam.-resist. | anti-gaming red-teamed | typed-graph action space |
|---|---|---|---|---|---|
| SWE-Bench [1] | code | ✓ | partial | – | – |
| $\tau$-bench [2] | tool use | ✓ | – | – | – |
| SWE-Gym [3] | code | ✓ | partial | – | – |
| classical NAS [11] | architecture | proxy | – | – | – |
| **ours** | **architecture** | ✓ | ✓ | ✓ | ✓ |

Table 8: Where this environment sits: architecture design as a verifiable-reward domain, with a contamination-resistant procedural generator, red-teamed anti-gaming defenses, and a typed-graph action space.

split is seed-addressable, a private seed produces reasoning data whose designs never appeared on the public web.

## 10 Related work

**Verifiable agent benchmarks.** SWE-Bench [1] and $\tau$-bench [2] established the programmatic-verifier pattern in code and tool use; SWE-Gym [3] showed that a few thousand verifiable tasks suffice to train agents to a double-digit absolute improvement on SWE-Bench, and did so as free, open research. We target the same pattern in a domain that lacked a public verifier (Table 8).

**RL environments and verifiable rewards.** Training on verifiable rewards (RLVR) is now central to reasoning-model post-training [4, 5], typically with policy-gradient methods such as PPO [7] and GRPO [6], preference optimization [9], or rejection sampling in the STaR [8] style. A growing market supplies labs with RL environments; reward-hacking resistance is the quality bar most consistently cited, and usable environments are expected to expose a hard band near a 2–3% pass floor. We build directly against both criteria, and ship GRPO and STaR loops against the same grader.

**Neural architecture search.** Classical NAS [11], from RL-based search [21] to regularized evolution [22], automates architecture design under a proxy objective, and NAS-Bench-101 [23] made such search reproducible by exhaustively tabulating a fixed cell space; our environment differs in providing a human-legible, per-edit, sub-millisecond verifier over a typed graph, and in serving as a training and evaluation substrate rather than a single search procedure. The accompanying product exposes a deterministic, constraint-satisfying design search (given a parameter budget, KV-cache budget, and target GPU, it returns the Pareto front over capacity, parameters, and KV cache), a NAS-lite special case built entirely from the verifier's physics rather than a learned proxy objective. Differentiable and one-shot NAS [10] make gradient search tractable by relaxing a small, hard-coded cell vocabulary and optimizing task loss; structural legality is assumed, not checked.

A per-edit legality verifier is orthogonal: it can veto the candidates of *any* search procedure – a DARTS derivation, an evolutionary mutation, or an LLM agent's edits – before GPU time is spent, so it composes with these methods rather than competing with them.

## 11 Limitations

The transparent rubric released here does not verify linear-width consistency (Section 5); the health-score magnitude is not a quality ranking (Section 5); the reward path does not execute PyTorch per rollout, so shape-class failures are caught statistically rather than per-instance; and generated references lean on `replace_model` for design-from-spec families, biasing imitation style. Each is documented with its measurement or its planned mitigation in `ROBUSTNESS.md`. The training experiments use one small policy (Qwen2.5-1.5B, LoRA) with evaluation splits of $n{=}192$ (headline) and $n{=}64$ (replications); the SFT/RL gains are largely in-distribution (out-of-distribution transfer to the curated split is suggestive, $3/12 \rightarrow 6/12$ with diverse training data), and contamination resistance is argued structurally rather than verified against a web-scale index. The amplification study, the near-miss audit, and the eleven-model blatant-tier audit were all re-run on generator v2 (diverse instances) with results matching v1, resolving the earlier effective-sample-size concern for those measurements. Scaling the policy and diversifying the minted data further are the natural extensions.

## 12 Conclusion

Representing architectures as typed graphs turns "is this network sound, and what will it cost to serve" into a pure function, and this paper measures what that function buys. As a repair signal, one round of its feedback takes a frontier model from $79\%$ to $100\%$ and a weaker one from $59\%$ to $90\%$. As a data foundry, the verified pairs it mints for free lift a small model from $17\%$ to $80\%$ on strictly-unseen tasks (SFT, then RL), under a train/eval-overlap audit we ran on ourselves, disclose, and ship. As ground truth, it exposes that LLM reward models with flawless records on blatant defects approve a quarter to a half of near-miss designs, failing precisely where reward hacking lives. We release the environment, the generator, the calibration and red-team harnesses, the training loops, and the public dataset, so that architecture design can join code and mathematics as a domain with a public verifier.

## Broader impact

The environment lowers the cost of designing valid, servable architectures and of training and evaluating agents that do so, which should broaden access beyond teams with dedicated ML-research staff. Two risks warrant note. First, a verifier defines what "good" means; ours checks structural validity and serving cost, not fairness, safety, or data appropriateness, and must not be mistaken for a quality oracle – the negative result in Section 5 is deliberate. Second, verified-data generation can accelerate capability gains; we mitigate by releasing the methodology openly, so the same tools that build also audit (the reward-model analysis above). The benchmark contains no personal data and no scraped content: every task is synthetic or hand-authored.

## Reproducibility

```
git clone <anonymized-repository>
cd arch-bench
npx vitest run                          # satisfiability + non-vacuity + anti-gaming
node calibrate.mjs --policy=reference   # harness self-test (must be 100%)
node calibrate.mjs --policy=noop        # harness self-test (must be 0%)
node training/build_sft_dataset.mjs \
     --count=3000 --seed=20260716      # mint verified pairs, holdout-excluded
python training/train_sft.py \
     --data arch-design-sft.chat.jsonl  # 17.2% -> 76.0% strictly-unseen (T4)
python training/train_grpo.py --eval-only --seed 999 --count 192 \
```

```
    --model out/sft-arch/checkpoint-final
python training/train_grpo.py --steps 100 --lora --lr 1e-5 \
    --model out/sft-arch/checkpoint-final  # SFT-then-RL -> 80.2%
node dump_grounding_set.mjs --count=40 --seed=123 --out=g.jsonl
python training/grounding_at_scale.py --set g.jsonl
python training/analyze_grounding.py
```

# A Example curated tasks

The twelve curated tasks are hand-authored from real production architectures. Figure 8 sketches four, as the typed graphs the environment operates on; each is a distinct family (vision, ranking, sequence, retrieval) with its own required layer types and budgets.

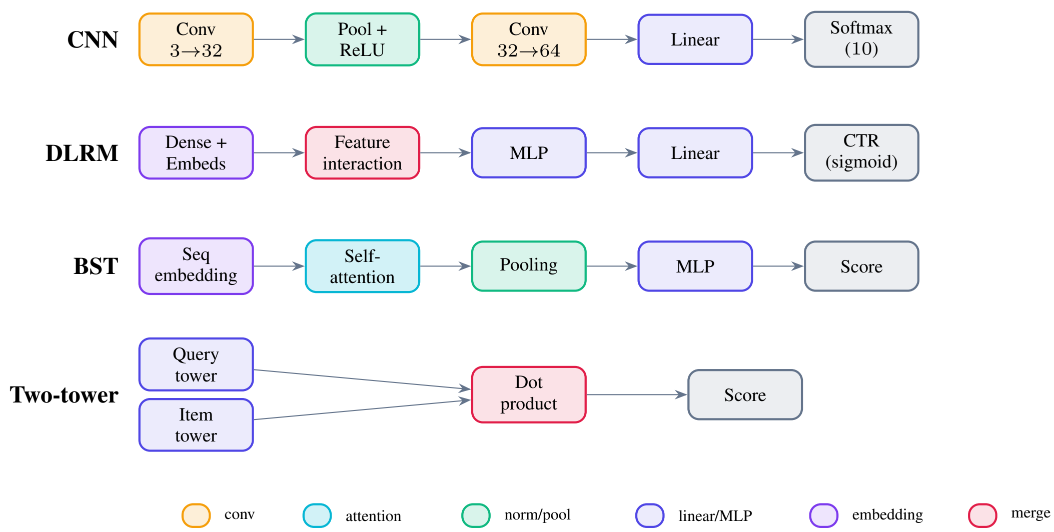


Figure 8: Four of the twelve curated tasks, as typed graphs, coloured by layer type (the same convention the accompanying production editor uses): a CIFAR-10 CNN, a DLRM click-through-rate ranker, a Behavior Sequence Transformer (which *requires* an attention layer), and a two-tower retrieval model (two encoders scored by a dot product). These are the states an agent edits and the verifier grades.

# References

[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR*, 2024.

[2] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. $\tau$-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv:2406.12045*, 2024.

[3] J. Pan, X. Wang, G. Neubig, et al. Training Software Engineering Agents and Verifiers with SWE-Gym. *arXiv:2412.21139*, 2024.

[4] N. Lambert et al. Tulu 3: Pushing Frontiers in Open Language Model Post-Training. *arXiv:2411.15124*, 2024.

[5] DeepSeek-AI (D. Guo et al.). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948*, 2025.

[6] Z. Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv:2402.03300*, 2024.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. *arXiv:1707.06347*, 2017.

[8] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman. STaR: Bootstrapping Reasoning with Reasoning. *NeurIPS*, 2022.

[9] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct Preference Optimization. *NeurIPS*, 2023.

[10] H. Liu, K. Simonyan, and Y. Yang. DARTS: Differentiable Architecture Search. *ICLR*, 2019.

[11] T. Elsken, J. H. Metzen, and F. Hutter. Neural Architecture Search: A Survey. *JMLR*, 20(55):1–21, 2019.

[12] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS*, 2023.

[13] L. Roeder. Netron: Visualizer for neural network, deep learning and machine learning models. Software, https://github.com/lutzroeder/netron.

[14] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, et al. Training Verifiers to Solve Math Word Problems. *arXiv:2110.14168*, 2021.

[15] E. B. Wilson. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[16] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.

[17] Qwen Team (A. Yang et al.). Qwen2.5 Technical Report. *arXiv:2412.15115*, 2024.

[18] N. Carlini, D. Ippolito, M. Jagielski, K. Lee, F. Tramèr, and C. Zhang. Quantifying Memorization Across Neural Language Models. *ICLR*, 2023.

[19] J. Skalse, N. H. R. Howe, D. Krasheninnikov, and D. Krueger. Defining and Characterizing Reward Gaming. *NeurIPS*, 2022.

[20] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's Verify Step by Step. *ICLR*, 2024.

[21] B. Zoph and Q. V. Le. Neural Architecture Search with Reinforcement Learning. *ICLR*, 2017.

[22] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le. Regularized Evolution for Image Classifier Architecture Search. *AAAI*, 2019.

[23] C. Ying, A. Klein, E. Christiansen, E. Real, K. Murphy, and F. Hutter. NAS-Bench-101: Towards Reproducible Neural Architecture Search. *ICML*, 2019.